Chapter 15 worked solutions • Security

OS15-E01 • Bounds and defense limits

1. Thirteen bytes are needed, one too many. Reject unless $n \leq 11$, or explicitly truncate to 11 data bytes and set buf[11]=0, reporting truncation. Copying 11 without NUL does not establish a valid string.
2. NX checks execution permission, not ordinary write bounds. Adjacent authorization data can be corrupted without executing stack code. Correct copy bounds. Data length, capacity, and terminator differ.

OS15-E02 • Pending-slot denial of service

1. $L=\lambda W$ gives $\lambda=2000/8=250$ requests/s. This mean model does not guarantee no instantaneous saturation below250; bursts/admission limits matter.
2. Users sharing NAT can share one quota, while an attacker can have many sources. Define key/fairness. Rate limiting does not promise harmlessness to authorized users. Units are requests/s, not bytes/s.

OS15-E03 • Reused keystream

1. c1=0xB5, c2=0xD5, c1 XOR c2=0x60=m1 XOR m2. Then s=c1 XOR m1=0xA5 and m2=c2 XOR s=0x70.
2. Associativity and s XOR s=0 prove leakage regardless of generator strength, without brute force. This toy demonstrates reuse, not production encryption.

OS15-E04 • Non-coprime RSA plaintext

1. $c=5^3 \bmod 55=15$. Squares are$15^2 \to 5, 15^4 \to 25, 15^8 \to 20, 15^{16} \to 15$. For$27=16+8+2+1$, multiply$15 \cdot 20 \bmod 55=25$, then $\cdot 5 \to 15$, then $\cdot 15 \to 5$.
2. Modulo5 the plaintext/power are0. Modulo11,5 is coprime and Fermat with $ed=1+2 \cdot 4 \cdot 10$ gives$5^{ed} \equiv 5$. CRT combines both. Euler modulo55 cannot be used under its violated coprimality condition. Correctness is not security of small RSA.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 15 worked solutions • Security (continued)

OS15-E05 • Hashes, MACs, and replay

1. An adversary replaces m with m′ and recomputes H(m′). A secret MAC resists new-message forgery but permits copying an old valid pair. Validate session/nonce/sequence state.
2. After valid tag verification, one lock/transaction checks pending and consumes it. Only one succeeds; separate checking/consumption lets both accept. Link business effects and deduplication atomically/recoverably too. Nonce consumption alone is not exactly-once external effect.

OS15-E06 • Historical TLS identity and secrecy

1. No. Validate name, purpose, chain, and other conditions; completing RSA exchange and verifying Finished establish master-secret derivation and transcript authentication. Forwarding another certificate requires no private key.
2. No. The recorded premaster can be decrypted with the later stolen static key; public randoms derive old master/traffic keys. Deletion does not add forward secrecy. This concerns RSA exchange, not every TLS version/suite.

OS15-E07 • Salt and guess cost

1. 500 • 20000=10000000 KDFs, times 0.002=20000 s≈5.556 h. Salt can be public beside the verifier; it separates computations rather than acting as a secret key.
2. No. Offline work bypasses online checks. Stated 2 ms cost still applies. Parallel hardware/memory assumptions are separate; serial modeled time is not measured attack time. Salt prevents easy cross-record table reuse without defeating dictionaries.

OS15-E08 • IDS base rate

1. Attacks100, normal99900; TP90, FN10, FP999, TN98901. Precision90/(90+999)=90/1089≈8.264%, not TPR90%.
2. Require πTPR≥(1−π)FPR, giving FPR≤0.001 • 0.9/0.999≈0.000900901, or0.09009%. Its denominator is normal events. Alarm precision and TPR use different denominators.

Chapter 15 worked solutions • Security (continued)

OS15-E09 • Firewall permissions

1. Deny, allow, deny, and allow only if the reply matches existing state. Port alone is insufficient; zones, addresses, direction, and state matter. Claiming established does not prove matching state.
2. Yes, through the permitted DB5432 path. Segmentation limits other entries without removing necessary trust. DB credentials, roles, query scope, and monitoring must constrain abuse.

OS15-E10 • Windows 7 DACL and MIC

1. For r, deny intersects no remaining bit, allow removes r, and DACL passes. NWU alone does not block read, so the model allows it. For rw, MIC rejects write-up; independently the DACL deny rejects w. Later allow does not override this rejection.
2. In noncanonical order, allow first empties remaining and DACL succeeds without scanning deny. MIC still rejects write-up. Neither all-denies-always-win nor DACL-success-implies-final-success is correct.
3. NULL has no DACL and permits under the ordinary check. Empty grants nothing, leaving requested bits and denying. Mandatory policy/other checks still apply. Keep each layer separate.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter 15 worked solutions • Security (continued)

Verification code appendix: security_models.py

```
"""Original finite arithmetic/state examples, not cryptographic deployment code."""
from fractions import Fraction
import hmac,hashlib
def precision(pi,tpr,fpr):return pi*tpr/(pi*tpr+(1-pi)*fpr)
def dacl(request,groups,entries):
 remaining=set(request)
 for kind,sid,mask in entries:
  if sid not in groups:continue
  if kind=='deny' and remaining&mask:return False
  if kind=='allow':remaining-=mask
  if not remaining:return True
 return False
if __name__=='__main__':
 assert pow(12,3,55)==23 and pow(23,27,55)==12
 assert [pow(15,2**i,55) for i in range(5)]==[15,5,25,20,15]
 assert all(pow(pow(m,3,55),27,55)==m for m in range(55))
 assert 0xE7^0xE4==0x41^0x42
 pr=precision(Fraction(1,10000),Fraction(9,10),Fraction(1,1000))
 assert pr==Fraction(100,1211)
 assert not dacl({'r','w'},{'U','G'},[('deny','G',{'w'}),('allow','U',{'r','w'})])
 assert dacl({'r','w'},{'U','G'},[('allow','U',{'r','w'}),('deny','G',{'w'})])
 key=b'local teaching
key';nonce=42;message=b'login|U|42|server';tag=hmac.digest(key,message,'sha256');pending=
{nonce}
 def accept():
  if hmac.compare_digest(tag,hmac.digest(key,message,'sha256')) and nonce in pending:
   pending.remove(nonce);return True
  return False
 assert accept() and not accept()
 print('RSA m12:',23,'->',12,'; all 55 toy messages checked (proof remains CRT)')
 print('IDS precision:',float(pr),'DACL order counterexample and sequential nonce replay
passed')
```

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study